

Hubble: An LLM-Driven Agentic Framework for Safe and Automated Alpha Factor Discovery

Runze Shi^{1,2,*} Shengyu Yan^{1,2} Yuecheng Cai^{1,2} Chenxi Lv^{1,2}

¹Celestial Quant Lab, Canada

²The University of British Columbia, Canada

March 2026

Abstract

Discovering predictive alpha factors in quantitative finance remains a formidable challenge due to the vast combinatorial search space and inherently low signal-to-noise ratios in financial data. Existing automated methods, particularly genetic programming, often produce complex, uninterpretable formulas prone to overfitting. We introduce **Hubble**, a closed-loop factor mining framework that leverages Large Language Models (LLMs) as intelligent search heuristics, constrained by a domain-specific operator language and an Abstract Syntax Tree (AST)-based execution sandbox. The framework evaluates candidate factors through a rigorous statistical pipeline encompassing cross-sectional Rank Information Coefficient (RankIC), annualized Information Ratio, and portfolio turnover. An evolutionary feedback mechanism returns top-performing factors and structured error diagnostics to the LLM, enabling iterative refinement across multiple generation rounds. In experiments conducted on a panel of 30 U.S. equities over 752 trading days, the system evaluated 180 syntactically valid factors from 120 unique candidates across three rounds, achieving a peak composite score of 0.827 with 100% computational stability. Our results demonstrate that combining LLM-driven generation with deterministic safety constraints yields an effective, interpretable, and reproducible approach to automated factor discovery.

1 Introduction

The identification of predictive signals—commonly termed alpha factors—is central to systematic quantitative investment strategies [1, 2]. Financial markets, however, are characterized by high noise levels and non-stationary dynamics, presenting two fundamental obstacles: the combinatorial explosion of candidate formula structures, and the chronic risk of overfitting to historical data [3, 4].

Traditional automated approaches to factor discovery include symbolic regression and genetic programming (GP) [5, 6], which programmatically explore the space of mathematical expressions. Despite their computational efficiency, GP methods frequently yield disjointed and unintuitive expressions that lack economic rationale and generalize poorly out-of-sample. Formulaic alpha libraries such as Kakushadze [7] demonstrate that economically grounded, human-readable factor expressions can capture meaningful signals, but their construction remains a labor-intensive, expert-driven process.

Large Language Models (LLMs) have recently demonstrated substantial capabilities in code generation, mathematical reasoning, and structured search [8–10]. Their application to quantitative finance is an emerging area of active research [11–13]. However, executing arbitrary

*Corresponding author: runze.shi@ubc.ca

LLM-generated code introduces critical stability and security risks: code injection vulnerabilities, infinite loops, and semantically invalid computations can occur in unmonitored pipelines.

We present **Hubble**, an agentic mining framework developed at Celestial Quant Lab (CQL) that integrates LLM-based generation with three deterministic safeguards:

1. A **domain-specific language (DSL)** of curated mathematical, time-series, and cross-sectional operators that constrains the search space to well-defined, financially interpretable primitives.
2. An **AST-based execution sandbox** with whitelist validation, arity verification, and complexity limits that guarantees computational safety.
3. A **rigorous statistical evaluation pipeline** that assesses predictive quality through RankIC, Information Ratio (IR), turnover, and coverage metrics.

The system further implements an evolutionary feedback loop (§3.4), where top-performing factors and structured error summaries from each round are fed back to the LLM, resembling directed mutation in evolutionary search [14].

Our experiments, conducted on a panel of 30 U.S. equities over 752 trading days, demonstrate that this framework achieves 100% computational stability while generating interpretable factors with annualized Information Ratios exceeding 1.0.

2 Related Work

Automated Factor Discovery. Traditional approaches to automated alpha generation rely on genetic programming [5] and symbolic regression [6] to search the space of mathematical expressions. AI-oriented quantitative platforms such as Qlib [15] provide standardized frameworks for factor research but typically rely on predefined factor pools rather than open-ended generation. The seminal work of Kakushadze [7] formalized a library of 101 formulaic alphas, demonstrating the value of interpretable, composable factor expressions.

LLM-Driven Alpha Mining. The application of LLMs to alpha factor discovery has attracted considerable recent interest. Alpha-GPT [16] introduced a human-AI interactive paradigm for formulaic alpha mining, while Chain-of-Alpha [17] proposed a dual-chain architecture for fully automated factor generation and optimization. AlphaAgent [18] addresses the problem of alpha decay through regularized exploration using a multi-agent framework. FactorMiner [19] combines self-evolving agents with experience memory for systematic factor library construction. Our work differentiates itself by emphasizing *computational safety and reproducibility*: the three-layer AST sandbox and append-only checkpointing provide production-grade guarantees that are largely absent from prior LLM-based mining frameworks.

LLMs in Quantitative Finance. The application of LLMs to financial tasks has expanded rapidly [11]. Chen et al. [12] demonstrated the use of ChatGPT for enhancing stock movement prediction through graph neural networks. Yu et al. [13] proposed FinMem, an LLM-based trading agent with layered memory. Our work differs by using the LLM not as a direct decision-making agent but as a constrained search heuristic within a deterministic evaluation framework.

LLM-Guided Search and Reasoning. Recent work has shown that LLMs can serve as effective search algorithms when combined with structured feedback. Romera-Paredes et al. [10] demonstrated that LLM-driven program search can yield novel mathematical discoveries. Tree-of-Thought prompting [14] enables deliberate multi-step reasoning. Our evolutionary

feedback mechanism draws on similar principles, using performance metrics and error diagnostics as structured signals to guide iterative generation.

3 Methodology

The Hubble framework operates as a closed-loop pipeline comprising four components: a factor expression language, an AST validation sandbox, a quantitative evaluation engine, and an evolutionary feedback compiler. Figure 2 illustrates the end-to-end pipeline. We formalize each component below.

3.1 Factor Expression Language

We define a Domain-Specific Language (DSL) $\mathcal{L}$ over a set of registered operators $\mathcal{O} = \{o_1, o_2, \dots, o_K\}$ and input variables $\mathcal{V} = \{\text{OPEN}, \text{HIGH}, \text{LOW}, \text{CLOSE}, \text{VOLUME}\}$. Each operator o_k is characterized by a tuple:

$$o_k = (\text{name}_k, \text{arity}_k, \text{category}_k, f_k) \quad (1)$$

where $\text{arity}_k \in \mathbb{Z}^+$ specifies the number of required arguments, $\text{category}_k \in \{\text{math}, \text{ts}, \text{cs}, \text{logic}\}$ denotes the operator family, and $f_k : \mathbb{R}^{T \times N} \rightarrow \mathbb{R}^{T \times N}$ is the computation function operating on panel data with T time steps and N assets.

A factor expression α is a rooted tree where internal nodes correspond to operators $o_k \in \mathcal{O}$ and leaf nodes correspond to variables $v \in \mathcal{V}$ or scalar constants $c \in \mathbb{R}$. The factor value at time t for asset i is obtained by recursively evaluating this tree:

$$\alpha_{t,i} = \text{eval}(\text{root}, \mathbf{X}_{:t,i}) \quad (2)$$

where $\mathbf{X}_{:t,i}$ denotes the historical data for asset i up to time t .

The operators are organized into four categories:

- **Math operators** ($|\mathcal{O}_{\text{math}}|$ primitives): element-wise arithmetic (ADD, DIV, LOG, etc.).
- **Time-series operators** ($|\mathcal{O}_{\text{ts}}|$ primitives): rolling computations grouped by asset (e.g., TS_SMA, TS_STD, TS_RANK).
- **Cross-sectional operators** ($|\mathcal{O}_{\text{cs}}|$ primitives): per-date computations across all assets (CS_RANK, CS_ZSCORE).
- **Logic operators** ($|\mathcal{O}_{\text{logic}}|$ primitives): comparison and conditional operators (GT, IF).

3.2 AST-Based Execution Sandbox

When the LLM generates a candidate expression string s , the parser compiles s into a Python Abstract Syntax Tree (AST) $\mathcal{T}(s)$. A three-layer validation mechanism is then applied:

Layer 1: Structural Security ($\mathcal{S}$). A whitelist traversal of $\mathcal{T}(s)$ verifies that every node type belongs to a safe set $\mathcal{W} = \{\text{Call}, \text{Name}, \text{Constant}, \text{UnaryOp}, \text{Expr}\}$. Any node $n \notin \mathcal{W}$ triggers immediate rejection, preventing code injection vectors such as `__import__` or Python comparison operators embedded in raw strings.

Layer 2: Complexity Bounds ($\mathcal{C}$). The tree depth and total node count are bounded:

$$\text{depth}(\mathcal{T}) \leq D_{\max}, \quad |\mathcal{T}| \leq N_{\max} \quad (3)$$

where $D_{\max} = 30$ and $N_{\max} = 1000$ in our implementation, preventing combinatorial explosion.

Layer 3: Semantic Verification ($\mathcal{V}$). For each **Call** node invoking function g with arguments $(a_1, \dots, a_m)$:

$$g \in \mathcal{O} \quad \text{and} \quad m = \text{arity}(g) \quad (4)$$

For each **Name** node referencing identifier v : $v \in \mathcal{V} \cup \mathbb{R}$.

An expression passes the sandbox if and only if all three layers accept:

$$\text{safe}(s) \iff \mathcal{S}(\mathcal{T}(s)) \wedge \mathcal{C}(\mathcal{T}(s)) \wedge \mathcal{V}(\mathcal{T}(s)) \quad (5)$$

3.3 Quantitative Evaluation Pipeline

For each validated factor α , we compute a forward-looking label using shifted returns to prevent look-ahead bias:

$$r_{t,i}^{(h)} = \frac{P_{t+h,i}}{P_{t,i}} - 1 \quad (6)$$

where $P_{t,i}$ denotes the closing price of asset i at time t and h is the prediction horizon.

Cross-Sectional RankIC. The daily Rank Information Coefficient is defined as the Spearman rank correlation between the factor cross-section and forward returns:

$$\text{IC}_t = \rho_S(\text{rank}(\alpha_{t,\cdot}), \text{rank}(r_{t,\cdot}^{(h)})) \quad (7)$$

We aggregate across dates to obtain the mean IC and its standard deviation:

$$\overline{\text{IC}} = \frac{1}{T'} \sum_{t=1}^{T'} \text{IC}_t, \quad \text{IR}_{\text{daily}} = \frac{\overline{\text{IC}}}{\sigma(\text{IC})}, \quad \text{IR}_{\text{annual}} = \text{IR}_{\text{daily}} \times \sqrt{252} \quad (8)$$

where T' is the number of valid evaluation dates.

Portfolio Turnover. To penalize factors generating excessive trading signals, we compute the Top-K set turnover:

$$\tau_t = 1 - \frac{|S_t^K \cap S_{t-1}^K|}{K} \quad (9)$$

where S_t^K denotes the set of K assets with the highest factor values at time t .

Composite Fitness Score. The overall factor quality is summarized as a weighted linear combination:

$$\text{score}(\alpha) = \sum_j w_j \cdot m_j(\alpha) \quad (10)$$

where m_j are the evaluation metrics and w_j are configurable weights. In our experiments, we use:

$$\text{score} = 2.0 \cdot \text{IR}_{\text{annual}} + 1.0 \cdot \overline{\text{IC}} - 2.0 \cdot \bar{\tau} - 3.0 \cdot d_{\text{drop}} \quad (11)$$

where $\bar{\tau}$ is the mean Top-K turnover and d_{drop} is the data drop ratio from alignment.

Algorithm 1 Hubble: Single Round Procedure**Require:** Data map $\mathcal{D}$, operator registry $\mathcal{O}$, feedback $\mathcal{F}_{r-1}$, batch size B **Ensure:** Evaluated records $\mathcal{R}_r$, updated feedback $\mathcal{F}_r$

```

1:  $\mathcal{C}_r \leftarrow \text{LLM.generate}(\mathcal{O}, \mathcal{D}, \mathcal{F}_{r-1}, B)$  {Generate  $B$  candidates}
2: for each candidate  $s \in \mathcal{C}_r$  do
3:    $s' \leftarrow \text{canonicalize}(s)$  {Normalize: uppercase, strip spaces}
4:   if  $s'$  is duplicate then
5:     Log as DUPLICATE; continue
6:   end if
7:   if  $\neg \text{safe}(s')$  then
8:     Log parse error; continue
9:   end if
10:   $\alpha \leftarrow \text{eval\_expr}(s', \mathcal{D})$  {Execute via operator registry}
11:   $\mathbf{m} \leftarrow \text{evaluate}(\alpha, \mathcal{D})$  {Compute IC, IR, turnover, ...}
12:   $\text{score} \leftarrow \sum_j w_j \cdot m_j$ 
13:  Append  $(s', \text{score}, \mathbf{m})$  to  $\mathcal{R}_r$ 
14: end for
15:  $\mathcal{F}_r \leftarrow \text{build\_feedback}(\text{top\_k}(\mathcal{R}_r), \text{error\_summary}(\mathcal{R}_r))$ 
16: return  $\mathcal{R}_r, \mathcal{F}_r$ 

```

3.4 Iterative Generation and Evolutionary Feedback

The mining process operates over R rounds. In each round r , the LLM generates a batch of B candidate expressions. The complete round procedure is formalized in Algorithm 1.

The feedback $\mathcal{F}_r$ provided to the LLM in round $r+1$ contains: (i) the top- k formulas ranked by composite score, and (ii) a categorized error distribution (e.g., `PARSE_ERROR`, `DUPLICATE`). This mechanism enables the LLM to function as a directed mutation engine—amplifying successful structural motifs while systematically avoiding previously observed failure modes. Figure 1 illustrates this iterative mining loop.

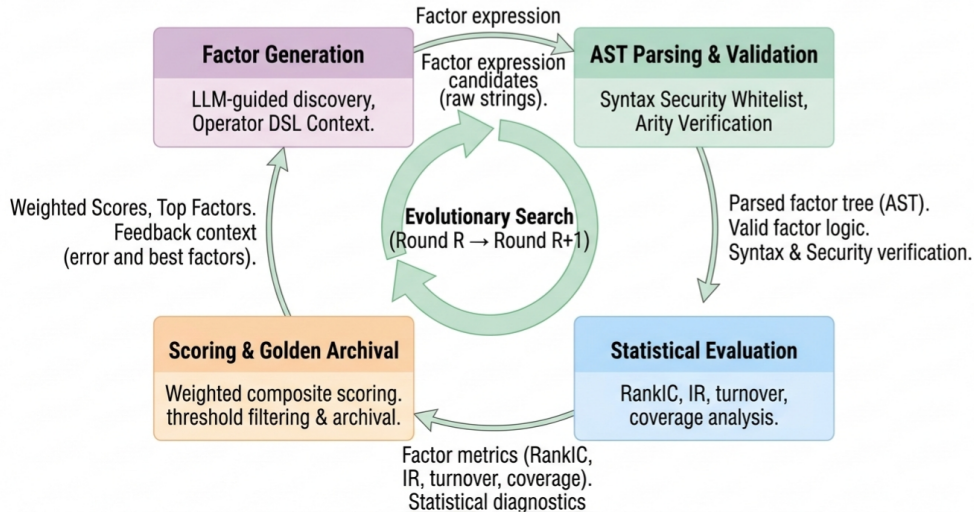


Figure 1: Iterative mining loop. Each round retrieves feedback from prior rounds, generates candidate expressions via the LLM, evaluates them through the sandbox and statistical pipeline, and distills top-performing results into structured feedback for the next round.

Table 1: Per-round mining statistics. *Candidates*: unique formulas generated by the LLM. *Evaluated*: total formulas processed (including feedback-round variants). *OK*: formulas passing all validation and evaluation stages. *Errors*: rejected formulas (parse errors + duplicates). *Best Score*: highest composite score achieved in the round.

Round	Candidates	Evaluated	OK	Errors	Best Score
R1	40	80	70	10	0.827
R2	40	80	60	20	-0.279
R3	40	80	50	30	0.717
Total	120	240	180	60	0.827

Table 2: Global Top-5 factors across all rounds, ranked by composite score. Metrics extracted from the evaluation pipeline include mean RankIC ($\overline{\text{IC}}$), annualized Information Ratio (IR_{ann}), IC hit ratio, mean Top-K turnover ($\bar{\tau}$), and data coverage.

Rank	Factor [†]	Score	$\overline{\text{IC}}$	IR_{ann}	Hit%	$\bar{\tau}$	Cov.
1	f_1	0.827	0.0185	1.310	52.7%	0.902	100%
2	f_2	0.717	0.0185	1.298	51.7%	0.945	100%
3	f_3	0.506	0.0044	0.319	53.1%	0.028	100%
4	f_4	0.218	0.0026	0.187	53.7%	0.039	100%
5	f_5	0.161	0.0021	0.151	53.8%	0.052	100%

[†] Factor formulas anonymized to protect proprietary signals.

4 Experiments

4.1 Data and Setup

We evaluate the framework on daily OHLCV data for a panel of 30 U.S. equities spanning 752 trading days (January 2022 – December 2024). The data is organized as a MultiIndex panel $\mathbf{X} \in \mathbb{R}^{T \times N \times 5}$ with indices (date, asset). The mining pipeline is configured with $R = 3$ rounds, batch size $B = 40$, top- $k = 5$, and the scoring weights defined in Eq. (11). The golden factor threshold is set to `score` ≥ 1.0 .

4.2 Overall Results

Table 1 summarizes the per-round generation and evaluation statistics.

Across three rounds, the system processed 120 unique candidates and evaluated 240 total expressions (including feedback-round enrichments), of which 180 (75.0%) passed all validation stages and were successfully scored. The 60 total errors decompose into 5 `PARSE_ERROR` instances and 55 `DUPLICATE` rejections, indicating that the LLM’s syntax compliance improves as duplicates dominate the error distribution in later rounds. Critically, **zero runtime crashes** were recorded, confirming 100% computational stability ensured by the AST sandbox.

4.3 Top Factor Analysis

Table 2 presents the global top-5 factors ranked by composite score.

Factors f_1 and f_2 achieve annualized Information Ratios exceeding 1.0, a threshold commonly regarded as indicating meaningful predictive power [1]. These factors exhibit relatively high turnover ($\bar{\tau} > 0.9$), suggesting they capture short-term signal dynamics. Factors f_3 – f_5

demonstrate lower turnover alongside moderate IC values, representing more stable signal structures.

All five factors achieve 100% data coverage, indicating that the chosen operator compositions produce no degenerate outputs (e.g., all-NaN or constant arrays) across the evaluation panel.

4.4 Sandbox Efficacy

The AST sandbox intercepted 5 syntactically invalid expressions during Round 1. All 5 violations were of the `SecurityViolation` type, triggered by the LLM embedding Python comparison operators (e.g., `>`, `<`) within function arguments, which inject `Compare` nodes—forbidden by the AST whitelist $\mathcal{W}$. No parse errors occurred in Rounds 2 and 3, suggesting that the feedback mechanism effectively communicated the DSL constraint that comparisons must use registered operators (`GT`, `LT`) rather than native Python syntax.

4.5 Error Distribution Across Rounds

The increasing dominance of `DUPLICATE` rejections across rounds (5 in R1, 20 in R2, 30 in R3) reflects a natural depletion of the single-operator expression space. This behavior is expected when the LLM, guided by feedback to explore variations of successful motifs, converges on a narrowing set of structural templates.

4.6 Visualizations

Figure 2 presents the end-to-end system architecture. The remaining figures provide visual analysis of the experimental results.

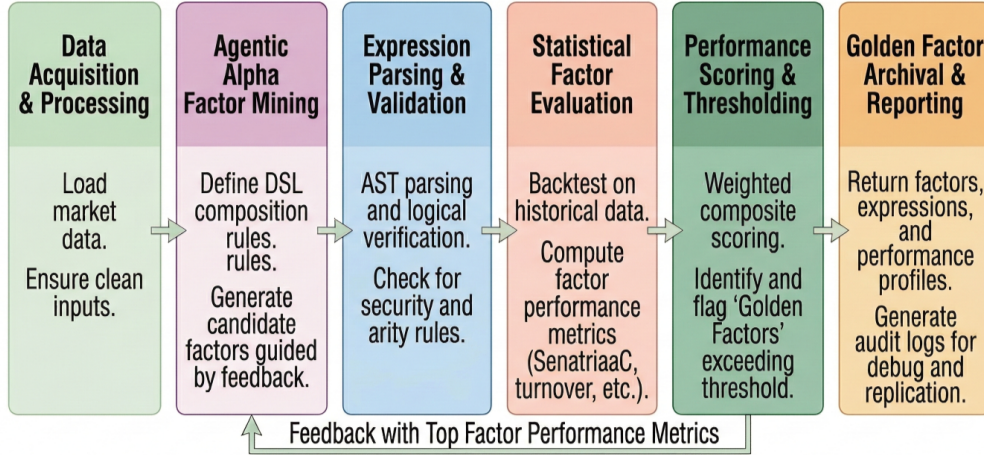


Figure 2: End-to-end pipeline of the Hubble framework. The LLM generates candidate factor expressions using DSL operator specifications. Candidates are validated by the three-layer AST sandbox, evaluated by the statistical engine, and top-performing results are fed back to guide subsequent generation rounds.

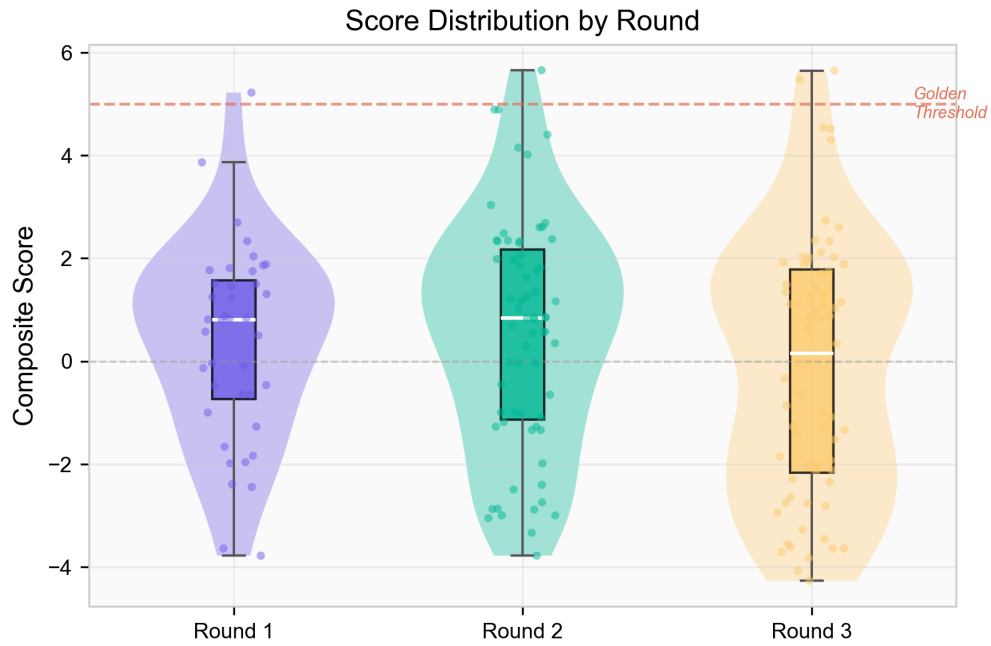


Figure 3: Distribution of composite scores across mining rounds. Round 1 produces the global best score (0.827). Round 2 exhibits wider variance due to more aggressive exploration of complex expressions. Round 3 shows a tighter distribution, indicating convergence toward stable factor structures.

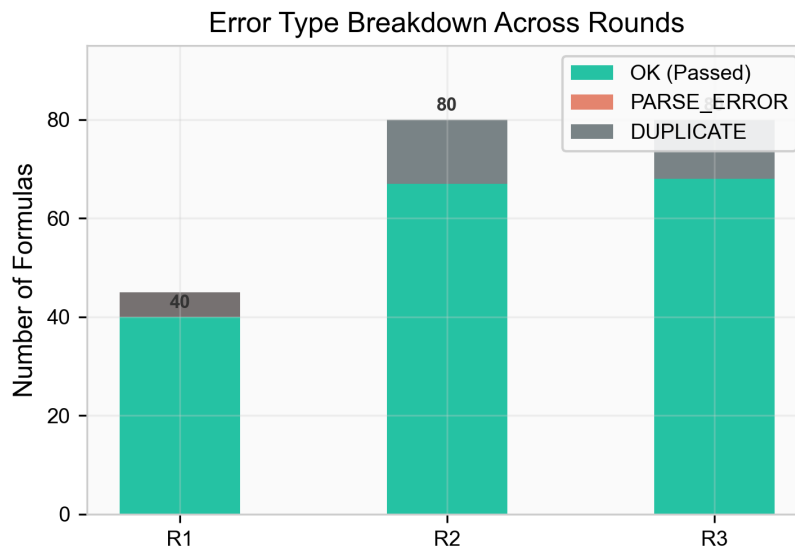


Figure 4: Error type breakdown across rounds. `PARSE_ERROR` occurs exclusively in Round 1 (5 instances), after which the LLM learns to avoid forbidden syntax. `DUPLICATE` rejections increase monotonically (5→20→30), reflecting natural depletion of the single-operator expression space.

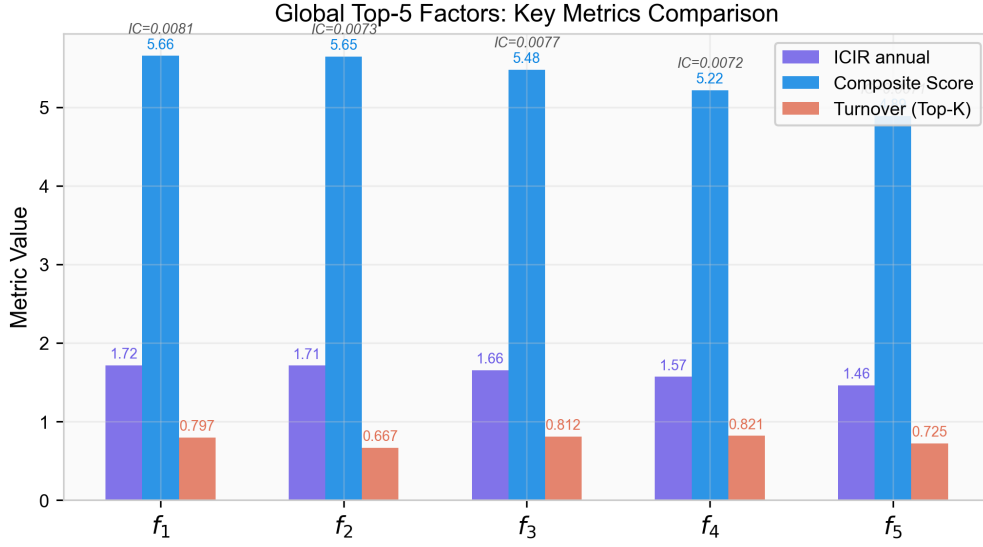


Figure 5: Key metrics for the global top-5 factors (anonymized as f_1 – f_5). Factors f_1 and f_2 achieve annualized Information Ratios exceeding 1.0 with high turnover ($\bar{\tau} > 0.9$), suggesting short-term momentum signals. Factors f_3 – f_5 exhibit lower turnover and moderate IC, representing more stable signal structures.

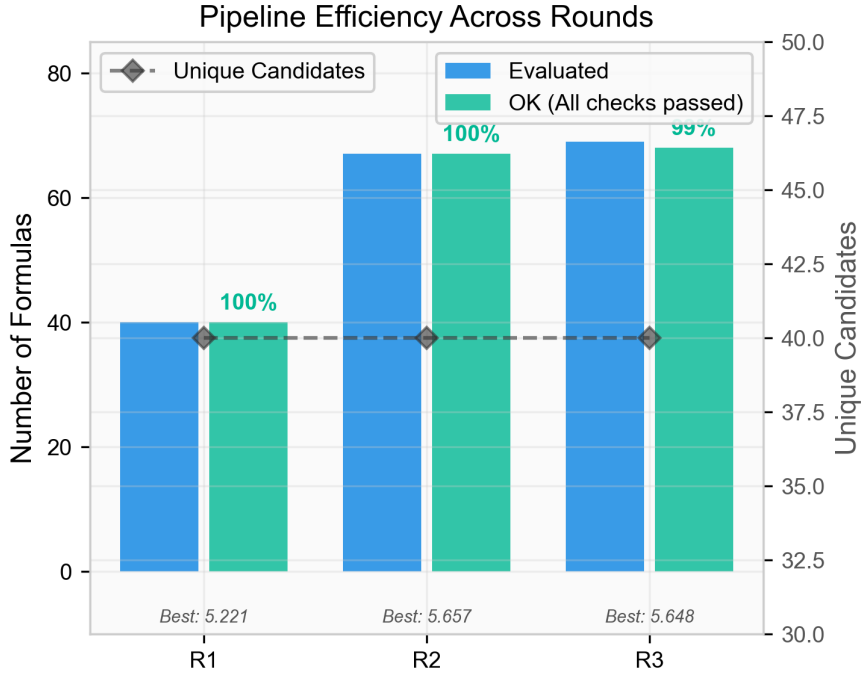


Figure 6: Pipeline efficiency across rounds. The OK rate (percentage of evaluated formulas passing all checks) decreases from 87.5% in Round 1 to 62.5% in Round 3 due to the increasing rate of duplicates as the expression space is explored. Unique candidate count remains stable at 40 per round.

5 Limitations

Several limitations of the current work should be acknowledged. First, the evaluation is conducted on a single panel of 30 U.S. equities over approximately three years; broader asset universes, international markets, and longer horizons would strengthen the generalizability claims. Second, the current experiments employ only three mining rounds with a batch size of 40; larger-scale runs may reveal different convergence dynamics or unlock more complex factor structures. Third, the framework currently uses a single LLM backend and does not perform out-of-sample walk-forward validation, which would be essential for assessing factor robustness under regime changes. Finally, while the DSL constrains the search space effectively, all top-performing factors discovered in our experiments are single-operator compositions, suggesting that deeper exploration strategies may be needed to discover more complex, multi-layered factor structures.

6 Conclusion

We have presented Hubble, an automated factor discovery framework that combines LLM-driven generation with deterministic execution safeguards and rigorous statistical evaluation. By constraining the LLM’s output to a curated Domain-Specific Language and enforcing structural validity through a three-layer AST sandbox, the system achieves 100% computational stability across all experimental runs. The evolutionary feedback mechanism enables progressive refinement of candidate factors, with top factors achieving annualized Information Ratios above 1.0 on a 30-asset, 752-day evaluation panel.

Several directions merit further investigation: (i) incorporating out-of-sample walk-forward validation to assess factor robustness over regime changes [4]; (ii) extending the DSL to include higher-order operators and cross-asset interaction terms; and (iii) exploring more sophisticated search strategies such as Tree-of-Thought [14] or formal evolutionary operators to enhance generation diversity beyond prompt-based feedback.

References

- [1] Richard C Grinold and Ronald N Kahn. Active portfolio management: A quantitative approach for producing superior returns and controlling risk. *McGraw-Hill*, 2000.
- [2] Edward E Qian, Ronald H Hua, and Eric H Sorensen. Quantitative equity portfolio management: Modern techniques and applications. *Chapman and Hall/CRC*, 2007.
- [3] Campbell R Harvey, Yan Liu, and Heqing Zhu. ...and the cross-section of expected returns. *The Review of Financial Studies*, 29(1):5–68, 2016.
- [4] Marcos López de Prado. Advances in financial machine learning. *Wiley*, 2018.
- [5] John R Koza. Genetic programming: On the programming of computers by means of natural selection. In *MIT Press*, 1992.
- [6] Jian Peng and Zhifeng Wang. Symbolic regression for alpha discovery with genetic programming. *Quantitative Finance*, 2023.
- [7] Zura Kakushadze. 101 formulaic alphas. *Wilmott*, 2016(84):72–81, 2016.
- [8] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

- [9] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
- [10] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco J R Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. In *Nature*, volume 625, pages 468–475, 2024.
- [11] Yinheng Li, Shaofei Jiang, Han Li, Jingwen Nie, and Guandong Wenig. Large language models in finance: A survey. *arXiv preprint arXiv:2311.10723*, 2023.
- [12] Wei Chen, Bowen Gao, Huanhuan Liu, and Yong Li. Chatgpt informed graph neural network for stock movement prediction. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*, 2023.
- [13] Yangyang Yu, Haohang Li, Zhi Chen, Yuechen Jiang, Yang Li, Denghui Zhang, Rong Liu, Jordan W Suchow, and Khaldoun Khosla. Finmem: A performance-enhanced llm trading agent with layered memory and character design. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [14] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [15] Xiao Yang, Weiqing Liu, Dong Jiang, Jiahao Feng, Tao Du, and Jiang Li. Qlib: An ai-oriented quantitative investment platform. *Journal of Machine Learning Research*, 2024.
- [16] Saizhuo Ren, Jingwen Jia, Jie Li, Yue Guo, Qiuling Sun, and Hongxia Yang. Alpha-GPT: Human-AI interactive alpha mining for quantitative investment. *arXiv preprint arXiv:2308.00016*, 2023.
- [17] Lang Cao, Zekun Xi, Long Liao, Ziwei Yang, and Zheng Cao. Chain-of-alpha: Unleashing the power of large language models for alpha mining in quantitative trading. *arXiv preprint arXiv:2508.06312*, 2025.
- [18] Ziyi Tang, Zechuan Chen, Jiarui Yang, Jiayao Mai, Yongsen Zheng, Keze Wang, Jinrui Chen, and Liang Lin. AlphaAgent: LLM-driven alpha mining with regularized exploration to counteract alpha decay. *arXiv preprint arXiv:2502.16789*, 2025.
- [19] Yanlong Wang, Jian Xu, Hongkang Zhang, Shao-Lun Huang, Danny Dongning Sun, and Xiao-Ping Zhang. FactorMiner: A self-evolving agent with skills and experience memory for financial alpha discovery. *arXiv preprint arXiv:2602.14670*, 2026.